

ECDAT - Complete Product Deep-Dive & Judge Defense Analysis

Prepared: 2026-09-04 - **Scope:** Full plain-English walkthrough of what ECDAT is, what every part of it does, why it exists, and what its real risks and limitations are.

How to read this document. This is written in theory and plain language, not engineering documentation. Nowhere does it quote code, file names, or technical implementation detail. Every claim in it is still grounded in a real, working part of the product - nothing here is invented or aspirational - but it is explained the way you would explain it to a person who needs to understand the product deeply, not read its source code. Where something is not built yet, this document says so directly rather than hiding it.

Table of Contents

1. Complete Architecture, in Plain English
 2. The Complete User Journey
 3. What Happens When You Scan a Repository
 4. The Cryptographic Discovery Engine
 5. How Classification Works
 6. Cryptographic Algorithm Inventory
 7. The Quantum Risk Engine
 8. The Quantum Vulnerability Model
 9. The Cryptographic Bill of Materials (CBOM)
 10. The Knowledge Graph Question
 11. The AI Analyst
 12. AI Ethics & Responsible AI
 13. Data Security & Data Leakage
 14. Authentication & Authorization
 15. Security Threat Model
 16. Technology Choices, and Why
 17. How the Core Algorithms Work, Conceptually
 18. Performance & Scalability
 19. Current Limitations - Stated Honestly
 20. Future Phases
 21. Standards & Compliance
 22. Judge Attack Mode - 50 Questions, Answered
 23. Hard Questions - Direct Answers
 24. Competitive Position
 25. The 5-Minute Demo Story
 26. Final "Know Your Product" Reference
 27. Technology Stack, in Detail
 28. How Each Cryptographic Algorithm Actually Works
 29. Core Terminology You Should Know
-

1. COMPLETE ARCHITECTURE, IN PLAIN ENGLISH

Think of ECDAT as three layers stacked on top of each other, plus a small number of outside services it talks to.

The layer the user sees is a web application. It is where a person logs in, connects a code repository, watches a scan run, and reads the results - risk scores, recommended fixes, a migration board, and a chat assistant. Nothing in this layer stores or decides anything permanently; it only displays what the layer beneath it has already computed.

The layer that does the work is a backend service. This is where every real decision gets made: cloning a repository, reading its code, deciding whether a piece of code uses cryptography, deciding which algorithm it is, calculating how risky that is under a quantum-computing threat model, recommending a replacement, and answering questions through the AI assistant. Every one of these is a distinct, independent piece of logic - there is no single "brain" making all these decisions; each is its own small, understandable rulebook.

The layer that remembers everything is a database. Every scan, every finding, every risk score, every recommendation, every migration status change, and every question ever asked of the AI assistant is stored here, tied permanently to the account that owns it. Nothing is ever silently deleted - if a scan finds the same problem twice, both findings are kept, because a security tool that can quietly lose evidence is not trustworthy.

Outside the three layers, ECDAT talks to four external services: an identity provider that handles login and proves who you are, two different AI providers that the assistant can fall back between if one is unavailable, and, implicitly, whichever code hosting service (like GitHub) hosts the repository being scanned - though ECDAT does not have a special relationship with any particular host; it simply asks for a public web address and downloads whatever is there.

What is deliberately not part of this system today: there is no separate graph database, no container or binary scanning subsystem, no dedicated job-queue infrastructure beyond a simple background-task mechanism, no multi-member organization or role system, and no automated deployment pipeline. These are discussed openly in the Limitations and Future Phases sections rather than glossed over.

2. THE COMPLETE USER JOURNEY

Before signing in

A visitor first sees a marketing-style landing experience explaining what the product is and why quantum computing is a real, near-term threat to today's cryptography. Nothing here touches real data - it exists purely to explain the problem. When the visitor decides to try the product, they are handed off to a hosted sign-in screen operated by the identity provider, which collects whatever it normally collects for a login (email and password, or a social login) - the product itself never sees or stores that credential; it only receives proof, afterward, that the person is who they say they are.

The very first login

The first time a person signs in, the system checks whether they already have a private "workspace" - the isolated space that will hold everything about their organization's scans, findings, and history. If they don't have one yet, they are shown a short setup screen and asked to create one. From this point on, everything they see is scoped entirely to that workspace - no other user, ever, can see into it.

The sixteen-plus screens, and what each one is actually for

The application is organized into six logical groups, shown as a sidebar. Below is what a person actually uses each one for, described the way you'd explain it to a new team member on their first day.

Command Center - *Mission Control* - the home screen. At a glance: how many projects are connected, how many cryptographic findings exist, how many are critical, and a single "Quantum Readiness" score summarizing the whole organization's posture. This is the screen an executive would look at once a week.

Discovery - *Sources* - where you register the code repositories you want scanned. You give it a name and a public web address; that's the entire setup. - *Scan Jobs* - where you trigger a new scan and watch its progress in something like a live log, so you're never left wondering whether it's stuck. - *Crypto Assets* - the master inventory. Every distinct cryptographic algorithm the system has ever found, across every scan, with its risk level and how many places it was found. - *CBOM Inventory* - an export of that same inventory in a standardized, industry-recognized document format, so it can be handed

to an auditor, a compliance tool, or another piece of security software.

Intelligence - *Dependency Graph* - a visual, rotatable 3D map showing which of your projects use which algorithms, so you can see at a glance whether one risky algorithm is spread across many projects or contained to one. - *Blast Radius* - for a single algorithm, this answers "if I have to change this, how much of my codebase does that actually touch" - how many projects, how many files, and what else lives alongside it. - *Evidence* - a single, searchable feed of every individual finding across the whole organization, down to the exact file and line it came from. - *Risk & Exposure* - the detailed risk breakdown per finding, plus a "what if" tool that lets you experiment with different assumptions (like "what if we had five more years to fix this") without changing any real data.

Quantum Transition - *Quantum Posture* - splits every finding into three honest buckets: algorithms that are completely broken by a quantum computer, algorithms that are only weakened (and just need a bigger key, not a total rebuild), and algorithms that are already safe. - *PQC Workbench* - the catalogue of quantum-safe replacement algorithms the system recommends, with the reasoning behind each recommendation. - *Migration Planner* - a five-column planning board (Assessed, Planned, In Development, Testing, Fully Migrated) that a team drags each finding through as they actually do the work of fixing it. This is described in full in the next section. - *Verification* - the proof step. This is what stops "we fixed it" from being a claim nobody checks: it re-scans the actual project and confirms, with real evidence, that the vulnerable algorithm is genuinely gone before anything is marked verified.

Analyst - *AI Analyst* - a chat-style assistant you can ask plain-English questions like "what should I fix first" or "which of my findings are quantum-vulnerable," and it answers using only the organization's own real findings - never a source-code file, and never a guess. - *Forecast & Labs* - reserved space for future experimental features; nothing live here yet.

System - *Activity* - a full audit trail of everything that has happened in the workspace: every scan, every setting change, every AI question asked. - *Compliance* - a live list of every finding that violates a basic security policy (an algorithm that should never be used at all, or one that needs review), plus a feed of alerts the system raises the moment it discovers something new and serious. - *Settings* - workspace configuration. Only one setting here is currently fully functional (the "threat horizon" assumption used by the risk calculation, described in Section 7); the rest of this screen currently displays example enterprise-style settings (team members, single sign-on, API keys, and so on) that are shown for context but are not yet backed by real functionality - discussed openly in Section 19.

How a person actually migrates something, start to finish

This is the heart of the product's value, told as a story:

1. A scan finds an outdated algorithm - say, an old hashing method used in a login system. It appears automatically in the **Assessed** column of the Migration Planner, with its real risk level already calculated.
2. The engineer opens the finding and sees exactly which files it lives in, the recommended modern replacement, and why that replacement was chosen. They drag it to **Planned** once they've decided to act on it.
3. As they start actually rewriting the code, they move it to **In Development**.
4. Once the code change is complete and undergoing their own testing, they move it to **Testing**.
5. Here is the important part: dragging a card to "Fully Migrated" by itself proves nothing - it only reflects what the person believes. So the system offers a **Verification** step: it re-runs a real scan of that exact project. If the old algorithm is truly gone, the system marks it "Verified" itself, based on fresh evidence, not on anyone's word. If the scan still finds it, the card stays exactly where it was, and the person is told plainly that the algorithm is still present.

This distinction - a human's claim versus the system's own evidence-based confirmation - is deliberate and repeated throughout the product. It is one of the more defensible design decisions in the whole system.

3. WHAT HAPPENS WHEN YOU SCAN A REPOSITORY

In plain terms, here is the full journey of a single scan, start to finish:

1. **You give it a repository address.** The system only accepts a normal public web address for a code repository - nothing more exotic.
2. **It downloads a working copy.** It takes only the most recent snapshot of the code, not the entire history of every change ever made to it - this keeps the download fast and avoids pulling in more than is needed.

3. **It puts that copy somewhere temporary and private.** This copy exists only for the few minutes the scan takes, in a location nothing else can reach, and is automatically and completely deleted the moment the scan finishes - whether the scan succeeded or failed.
4. **It reads the code using several independent methods at once** (explained fully in Section 4) - looking for imports of cryptographic libraries, calls to cryptographic functions, cryptographic packages listed as dependencies, and any certificate files sitting in the repository.
5. **It records only what it found, not the whole file.** For each match, it keeps the specific line that matched and a couple of lines of surrounding context for readability - never the entire file, and never anything from files that had no match at all.
6. **It cleans up completely.** The temporary copy of the repository is deleted immediately afterward, every time, even if something went wrong partway through.
7. **It turns raw findings into a clean inventory,** calculates risk for each one, generates recommendations, and updates the organization's exported compliance document - all automatically, with no further action needed from the person who triggered the scan.

Direct answers to the questions people always ask about this step

- **Is the whole repository copied and kept?** No - only the newest version is downloaded, and only for the few minutes the scan takes; nothing is retained afterward.
- **Is any actual source code stored permanently?** No. Only the specific matched lines relevant to a cryptography finding are kept - and only if a finding actually exists there. A file with no crypto in it leaves no trace.
- **Who can see the temporary copy while it exists?** Nobody outside the scanning process itself - it isn't exposed anywhere, isn't served over the web, and exists on a machine only the system itself has access to.
- **What happens to secrets, private keys, environment files, or certificates found inside the repository?** Certificate files are actively read for their public information (what algorithm and key size they use, who they belong to, when they expire) - but never for the private key material itself, which is never extracted or stored. Environment files and other secret files are not specially treated at all; they would only be noticed if their contents happened to resemble a cryptography finding, and even then only the matching line would be kept.
- **What about compiled programs (binaries)?** They are invisible to the system today - binary analysis is a capability the product does not yet have (see Section 19).
- **What if the repository is malicious?** The code inside a repository is only ever read, never executed or run - so a hostile repository has no direct way to make the scanning system do anything other than read its text.
- **What if a file is enormous?** There is currently no safeguard against an unusually large file slowing down or straining the scan - this is a real, acknowledged limitation, not something already handled.
- **What if the repository contains actual malware?** The product is not an antivirus and makes no attempt to detect malware - that is genuinely outside its purpose.
- **Could something hidden in the code manipulate the AI assistant later (a "prompt injection" attack)?** No - because the AI assistant is never shown actual source code text at all (explained fully in Section 11), there is no path for text hidden in a repository to reach it.
- **What about personal or sensitive information sitting in the code?** It isn't specifically searched for or protected - if it happens to sit on the same line as a cryptography finding, that line (not the whole file) would be retained the same way any other finding is.

4. THE CRYPTOGRAPHIC DISCOVERY ENGINE

ECDAT does not use artificial intelligence to find cryptography in code. It uses four separate, completely predictable methods, each suited to a different kind of evidence:

1. **Reading the actual structure of the code.** Rather than just searching for words, the system properly parses each source file the way a compiler would, so it understands the real difference between an actual cryptographic function call and, say, a comment or an unrelated word that merely looks similar. This method understands seven common programming languages.

2. **A curated set of pattern-matching security rules.** A second, independent method checks for more specific patterns - for example, not just "is RSA used" but "is RSA being used with a key size that's already too small even by today's standards."
3. **Reading project dependency lists.** Many programming ecosystems declare which external libraries a project uses in a simple manifest file. The system checks these manifests against a known list of cryptography-related packages, which is a fast and highly reliable way to catch cryptography used through a library rather than written by hand.
4. **Reading digital certificates.** If the repository contains any certificate files, the system genuinely parses them and extracts their real cryptographic properties - what algorithm signed them, what key size they use, and when they expire.

Why not just use AI to find cryptography? Because a discovery tool's most important property is that it finds the *same* things every single time it looks at the *same* code, and that it doesn't miss things simply because a sentence was phrased unusually. A language model is well-suited to explaining and summarizing findings in natural language - it is not well-suited to being the source of truth for "does this specific line of code use RSA," because it can be inconsistent, and a security tool that quietly misses real findings is worse than one that is merely slow. This is why every actual detection decision in ECDAT is made by a predictable, repeatable method, and the AI is only ever layered on top of results those methods have already produced.

Honest limitations of this approach, stated directly: - It can only recognize cryptography that matches a known pattern - a genuinely custom, hand-built cipher with no recognizable structure would not be detected. - It only understands the languages it has been taught to read - other languages are invisible to it. - It cannot see cryptography that's hidden behind a layer of abstraction (for example, a company's own internal helper function that calls a cryptographic library indirectly) unless that inner call itself is visible somewhere. - False positives and false negatives are a known, expected reality of any pattern-based detector; several real examples of both were found and fixed during development, which is a healthy sign of active testing - but no formal, published accuracy rate (like "95% of findings are correct") currently exists for this system.

5. HOW CLASSIFICATION WORKS

Once something has been flagged as "probably cryptography," a second step decides exactly *what* it is - "this is RSA," "this is SHA-256," and so on.

This step works entirely by **lookup**, not by inference or guesswork. The system holds a curated reference list of known cryptographic algorithm names and their common spelling variations, and it matches whatever text it found against that list. If nothing matches, the system does *not* guess or invent a name - it simply records the raw finding as unclassified evidence rather than pretending to know what it is. This is a deliberate honesty choice: an unnamed finding is more trustworthy than a confidently wrong one.

From there, deciding "is this quantum-vulnerable" or "is this already broken today" is also a lookup, not a judgment call - the system checks the algorithm's name against two curated reference lists: one of algorithms known to be completely broken by a quantum computer, and one of algorithms already considered broken by today's ordinary computers.

Are confidence scores calculated statistically? No, and this is worth being precise about. Every finding is assigned a confidence number, but that number is a fixed value tied to *which detection method* found it (for example, a certificate reading is always treated as fully certain, while a looser code-pattern match is treated as slightly less certain) - it is not a probability computed individually for that specific finding. If a judge asks "how was this exact number calculated," the honest answer is that it reflects the general reliability of the method used, not a per-finding statistical calculation.

Is any of this driven by machine learning or a language model? No. Every step described in this section is a deterministic rule or lookup - the same input always produces the same output, with no learning, training, or probability involved anywhere in it.

6. CRYPTOGRAPHIC ALGORITHM INVENTORY

This is the full list of algorithm families the system understands, what it knows about each one, and what it recommends instead.

Public-key algorithms (used for secure key exchange and digital signatures)

Algorithm	Why it matters	Quantum status	Recommended replacement
RSA	The most widely used public-key algorithm in the world	Completely broken by a quantum computer, regardless of key size	A quantum-safe key-exchange algorithm, or a quantum-safe signature algorithm, depending on how RSA was being used
DSA	An older digital-signature algorithm	Completely broken	Quantum-safe signature algorithm
ECDSA / ECDH (and their common curve variants)	Modern, compact public-key algorithms, very widely used	Completely broken	Quantum-safe signature or key-exchange algorithm
Diffie-Hellman	A classic key-exchange method	Completely broken	Quantum-safe key-exchange algorithm

Symmetric algorithms (used to actually encrypt data)

Algorithm	Quantum status	Recommendation
AES with a small key	Weakened, but not broken - effectively loses about half its strength	Upgrade to a larger key size
AES with a large key	Still considered safe	No change needed
DES / Triple-DES	Already broken by ordinary computers today, independent of quantum computing	Replace with a modern authenticated cipher
Blowfish	Has a known structural weakness in how much data it can safely encrypt under one key	Replace with a modern authenticated cipher
RC4	Already broken by ordinary computers today	Replace with a modern authenticated cipher

Hashing algorithms (used to fingerprint data, e.g. for passwords or integrity checks)

Algorithm	Status
MD5, MD2	Already broken by ordinary computers today
SHA-1	Already broken by ordinary computers today
SHA-256 and larger	Currently considered acceptable and safe, including against quantum weakening

Key-derivation and message-authentication algorithms

The system does not currently have dedicated recognition for this entire category - things like password-hashing functions or message-authentication codes. This is a genuine, acknowledged gap (Section 19), since these are extremely common in real password-security code.

Certificates

The system can genuinely read a digital certificate and tell you what algorithm signed it, what key size it uses, and its validity dates. It does not currently inspect a live web server's encrypted connection directly - only certificate files that are physically present in the repository being scanned.

Post-quantum (the recommended replacements)

The system recommends a small, current set of algorithms that have been formally standardized specifically to resist quantum attacks - one family for key exchange, one for digital signatures, and a more conservative backup signature option for organizations that want an extra layer of caution. It also supports "hybrid" recommendations - running an old and a new algorithm together during a transition period, so nothing breaks compatibility while the migration is underway.

7. THE QUANTUM RISK ENGINE

This is the part of the product that turns a raw finding ("we found RSA here") into a business-relevant risk level ("this is critical, fix it now").

The idea in plain English

The risk calculation is built around a simple, well-known piece of reasoning used across the security industry, sometimes called "Mosca's inequality." It asks three questions about any piece of encrypted data:

- **How long does this data need to stay secret?** (Some data only matters for a few months; medical or financial records might need to stay confidential for decades.)
- **How long would it realistically take us to actually migrate this to a safe algorithm?** (Some fixes are quick; replacing a deeply embedded public-key system across a whole company can take years.)
- **How much time do we realistically have before a quantum computer capable of breaking today's cryptography exists?**

The core insight is this: if the time your data needs to stay secret, plus the time it will take you to migrate, together exceed the time you actually have left - you are already at risk *today*, even if no quantum computer exists yet. This is because an adversary can copy and store your encrypted data right now, and simply wait to decrypt it later once the technology catches up - a real and widely discussed threat known as "harvest now, decrypt later."

The system takes these three numbers, does this comparison, and produces one of four risk levels - from "no urgency yet" through to "the danger window is already open, act immediately."

How the final priority is actually decided

The quantum-based calculation above is only part of the picture. If something is *already* broken by ordinary means today (like an old, cracked hashing algorithm), that alone is treated as the most urgent kind of risk, regardless of quantum timing - because that is an active threat right now, not a future one. Quantum risk only comes into play for algorithms that are actually vulnerable to a quantum computer in the first place; something that's neither quantum-vulnerable nor already broken today is never artificially bumped up in urgency just because of a generic organization-wide timeline.

Is this calculation scientifically "correct"?

This deserves an honest, direct answer rather than a confident one. The *reasoning method itself* - comparing data lifetime plus migration time against the time remaining - is a well-established, widely respected way of thinking about this problem, and the system implements that comparison faithfully and consistently. However, two of the three input numbers are **estimates, not measurements**: how long a migration typically takes for a given type of algorithm is a reasonable, named assumption, not something derived from real observed data; and the assumed number of years remaining before a capable quantum computer exists is likewise an editable estimate, openly labeled as such, not a scientific prediction. The system deliberately makes that second number something an organization can adjust themselves, precisely because it's understood to be a judgment call, not a fact. So: the *math* is sound and consistent; the *inputs feeding the math* are reasonable estimates that should be understood as such, not treated as certainties.

8. THE QUANTUM VULNERABILITY MODEL

It's important to understand that quantum computers don't threaten every kind of cryptography the same way - and the system is careful to reflect this correctly rather than treating "quantum risk" as one single, undifferentiated danger.

- **Algorithms based on public-key mathematics (like RSA and elliptic-curve cryptography) are not weakened by quantum computing - they are completely broken by it.** A sufficiently powerful quantum computer can solve the underlying mathematical problem these algorithms depend on directly, which means no key size, however large, would save them. This is the most urgent category, and it is what most conversations about "post-quantum" migration are actually about.
- **Algorithms based on symmetric keys or hashing (like AES or SHA-256) are only weakened, not broken.** A quantum computer roughly halves how strong these feel in practice - meaning a previously adequate key size might no longer be enough, but the fix is simply to use a bigger key, not to abandon the algorithm entirely.
- **"Harvest now, decrypt later"** describes the specific danger of encrypted data being captured and stored today by an adversary, to be decrypted retroactively once quantum computers mature enough - meaning the risk clock for some data is already running, even though no such computer exists yet.

The system's risk model correctly keeps these two categories - "completely broken" versus "just weakened" - separate throughout, both in how it calculates urgency and in how it explains the risk to a user (recommending an entirely new algorithm for one category, and simply a bigger key for the other). This distinction matters and is one of the places the product is most careful to get right.

Where this is simplified: the model treats "this algorithm is weakened" as a fixed, permanent classification today, rather than something that changes gradually as quantum computers actually improve over time - a more advanced version of this model would tie that classification to the same "how many years do we have left" assumption used elsewhere in the risk calculation.

9. THE CRYPTOGRAPHIC BILL OF MATERIALS (CBOM)

Think of a CBOM the same way you'd think of a nutrition label, but for the cryptography inside a piece of software - a standardized, structured, exportable list of exactly which cryptographic "ingredients" are present.

Why this exists, rather than just a table in the dashboard: because it needs to be usable *outside* the product too - handed to an auditor, imported into a compliance system, or compared against a previous export to see what changed over time. ECDAT generates this in a recognized industry-standard document format, in two common file formats, so it fits into whatever tooling an organization already uses.

What's inside it: every distinct algorithm the system has ever found, its type (a hash, a cipher, a signature algorithm, and so on), its key size where known, and whether it's flagged as quantum- or classically-vulnerable - along with a permanent unique identifier so the exact same algorithm is always represented as the exact same entry, never duplicated.

How duplicates are handled: if the same algorithm is found in ten different files across three different projects, it still appears as *one* entry in the inventory - the underlying evidence (which files, which projects) is tracked separately and linked to that one entry, rather than the inventory being cluttered with ten repeated rows.

What happens when the system genuinely doesn't know what something is: nothing gets fabricated. If a raw finding can't be confidently matched to a known algorithm name, it is simply not turned into an inventory entry at all - the underlying evidence is still kept, just not dressed up as a named, classified asset.

History: every time a scan runs, a brand-new, complete snapshot of the whole inventory is saved - so an organization can look back and compare "what did our cryptographic footprint look like three months ago" against today.

Standards compliance: the export follows the structure of a well-known, industry-recognized standard closely, and the system checks its own output against the key rules of that standard before producing it. That said, this is a self-check the system performs itself - it has not been independently certified or validated by an outside authority, and that distinction should be stated plainly rather than implied away.

10. THE KNOWLEDGE GRAPH QUESTION

This is worth addressing head-on, because it's one of the areas where it would be easy to overstate what exists.

There is no dedicated graph database in this product today. What the "Dependency Graph" screen actually shows is a genuine, real-data visualization - built from the organization's actual scan results - that draws a simple, two-level picture: which projects use which algorithms. It is presented as an interactive, rotatable 3D view because that makes the real relationships easier to explore visually, but underneath, it is not powered by specialized graph-database technology, and it does not model anything beyond that one level of relationship.

The "Blast Radius" screen is the more precise and more honestly-labeled version of this same idea: for one specific algorithm, it tells you exactly how many projects and files it touches, and what else lives alongside it in those same projects - genuinely computed from real data, but explicitly presented as a lighter-weight view, not a full enterprise dependency map. The system itself is upfront that this is not the same thing as a true multi-layer graph connecting, say, individual applications to business systems to the data they handle.

What a full version of this would eventually add: the ability to trace a finding not just to "which project" but through several more layers - which specific application, which business system depends on that application, and ultimately what real-world data or business process is affected - enabling much richer "if I fix this, here's everything downstream that's affected" analysis. That deeper capability does not exist yet and is realistically future work, not a current feature.

11. THE AI ANALYST

Where AI is used, and - just as importantly - where it is deliberately not used

To be direct about this: **artificial intelligence plays no role anywhere in finding cryptography, naming an algorithm, calculating a risk score, generating a recommendation, or producing the compliance export.** Every one of those steps, described in the sections above, is handled by predictable, rule-based logic with no AI involved at all. The **only** place AI is used in the entire product is in one specific feature: answering a person's typed, natural-language question.

How a question actually gets answered

When someone asks the assistant a question, here is what genuinely happens, step by step, in plain terms:

1. The system first gathers a curated, ranked summary of the organization's own real findings - prioritized by how risky they are - along with each finding's risk level, recommended fix, and *where* it was found (which file, which line, which project). Critically, it does **not** include the actual matching source code text itself in this summary - only the location and metadata around it.
2. This real, factual summary - and only this - is handed to the AI model, along with strict instructions: answer only using this data, never invent a finding or a risk level that isn't in it, and be explicit whenever the question asks about something the data doesn't cover.
3. The system is built to use one AI provider by default, and automatically fall back to a second, independent provider if the first one is temporarily unavailable for any reason - so a single AI outage doesn't take the whole feature down. This exact fallback was tested for real during this project's own development, when the primary provider hit a temporary usage limit and the backup provider stepped in successfully and transparently.
4. Once the AI responds, the system does not simply trust it. Every specific fact the AI claims to be citing - a specific finding, a specific piece of evidence - is independently checked against the organization's real database before being shown to the user. If the AI ever refers to something that doesn't actually exist in the real data, that reference is silently removed rather than shown as if it were real. This is the core of how the system protects against the AI simply making something up.
5. If the AI response doesn't come back in the expected, structured shape at all, the system rejects it outright and tells the user honestly that something went wrong, rather than risk showing a malformed or unreliable answer.
6. Every question and its full answer is saved as part of a permanent, reviewable conversation history, and every question asked is also separately logged in the organization's audit trail.

A concrete example of how a recommendation actually reaches the user

Say an old, insecure key-exchange algorithm is found inside a payment system. The decision that this is dangerous, and how dangerous, is made entirely by the deterministic risk engine described in Section 7 - long before anyone asks the AI anything. If a person later asks the assistant "what should I fix first," the assistant is simply looking at a list that has *already* been ranked by that risk engine, and describing the top of that list back to the person in natural language. The AI is narrating an already-made decision - it is not the one making it.

What this means for trust

- The AI **cannot** change a risk score, alter a recommendation, or mark anything as migrated or verified - it has no ability to write to the organization's real data at all, only to read a summary of it and hold its own separate conversation history.
- Actual source code is never sent to the AI provider - only a location-and-summary description of a finding, never the matching text itself.
- Every citation the AI makes is checked against real data before a user ever sees it.

Where this is honestly incomplete: while every specific fact the AI cites is checked, the general prose of its answer is not independently re-verified word-by-word - so it's possible, in principle, for a sentence to be phrased in an unhelpful or slightly imprecise way even though every concrete fact and citation within it is accurate. This is a real, acknowledged gap, not a claim of perfection.

12. AI ETHICS & RESPONSIBLE AI

Privacy - Is source code ever sent to an outside AI service? No. Only a summary describing *where* a finding is and *what kind* of risk it carries is ever included - never the actual matching code text. This is a structural guarantee built into how the assistant's input is assembled, not just a policy promise.

Transparency - Every answer shows which project(s) it's drawing from, a confidence indicator, and clickable references back to the real evidence behind each specific claim.

Explainability - Every specific fact the assistant cites is grounded in, and checked against, real evidence a person can click through and verify themselves - it is not a black box making unexplained claims.

Human oversight - Nothing the AI says can, by itself, change any real risk score, recommendation, or migration status. A human being always has to take the actual action (drag a card, click a button, or trigger a real re-scan) for anything to actually change. The AI's role is strictly advisory.

Accuracy and hallucination control - Every specific citation is independently checked against real data and silently dropped if it doesn't hold up; malformed or improperly structured responses are rejected outright rather than shown. The general prose surrounding those citations is not independently fact-checked sentence by sentence, which is an honest, stated limitation rather than a hidden one.

Bias - Because the ranking of what's most urgent is decided entirely by the deterministic risk engine *before* the AI is ever involved, the AI has no ability to systematically favor or deprioritize any particular kind of finding - it can only describe a ranking it didn't create.

Security - Because actual source code text never reaches the AI, a common attack pattern where hidden text in a document tries to manipulate an AI system's behavior has no path into this feature at all.

Accountability - Every single question asked of the assistant is permanently logged, both in a full conversation history and separately in the organization's audit trail, so every AI interaction that has ever happened in a workspace can be reviewed later.

AI Ethics Readiness Score: 7 out of 10

Why: Strong, genuinely structural protections exist - no source code ever leaves the system, the AI has zero ability to alter real data, every specific citation is independently verified, every interaction is logged, and organizations can exclude specific sensitive projects from AI visibility entirely. Points are held back because the *prose* of an answer isn't independently fact-checked beyond its citations, and because what happens to the (non-code) information after it reaches

an outside AI provider's own systems is governed by that provider's own policies, not something this product can fully control or verify on its own.

13. DATA SECURITY & DATA LEAKAGE

How is a repository actually accessed? Only through a plain public web address - there is currently no support for connecting a private repository through a login or access-token style connection. One direct consequence of this is that there are simply no repository access credentials anywhere in the system to protect or leak, because none are ever collected or stored in the first place. The tradeoff, stated plainly, is that private repositories cannot currently be scanned at all.

Data at rest - All real data lives in a managed, professionally-hosted database service, rather than a self-managed server the product's own team operates directly - meaning baseline protections like encryption of stored data are handled by that hosting platform. Nothing is stored in any separate file-storage system, because the product doesn't use one. The temporary copy of a repository made during a scan exists only briefly, in a private, isolated location, and is fully deleted immediately afterward.

Data in transit - Communication between the parts of the system uses standard secure web connections; the exact security of the final, deployed network path depends on how and where the product is ultimately hosted, which is outside what the application's own code can fully control or guarantee on its own.

What data reaches the AI providers, and what happens to it there - As described in Section 11, only finding summaries (never code) are sent, and only for findings relevant to whatever question is being asked. What each AI provider does with that data afterward on their end is governed by their own policies, not by this product - a fair and honest thing to say plainly rather than promise more than can be guaranteed.

Secret keys and credentials - The handful of secret values the system itself depends on (like the keys it uses to talk to the identity provider and the AI services) are kept in a configuration file that is deliberately excluded from the project's shared code history, so they are never accidentally published alongside the rest of the source code - this was specifically double-checked and confirmed during this review.

How is one organization kept completely separate from another? Every single piece of data in the system - every finding, every risk score, every conversation with the AI, every setting - is permanently tied to the one workspace it belongs to, and every single request the system ever processes is checked against who is actually logged in before any data is returned. This check happens consistently, everywhere, including for the more unusual cases (like looking up one specific finding or one specific past AI conversation directly) - not just for the obvious, everyday screens. In practice, this means a user genuinely cannot see another organization's data, even by guessing or manipulating a web address, because ownership is always re-confirmed against the real, verified identity of whoever is asking - never simply assumed from what the request claims.

14. AUTHENTICATION & AUTHORIZATION

Who handles login? A dedicated, specialized identity provider - not something the product built itself. This is a deliberate choice, since identity and login security is a deep specialty best left to a provider whose entire business is getting that right.

How does the backend know a request is really from a logged-in user, and not faked? It doesn't simply trust whatever the browser tells it. Every single request is independently re-verified against the identity provider's own cryptographically signed proof of login - meaning even if someone tried to forge a request claiming to be a different user, that forgery would be rejected, because the backend checks the *cryptographic signature* behind the claim, not just the claim itself.

Are there teams, roles, or admin permissions? Not yet - today, the system is built around a strict one-person-per-workspace model. There is no concept yet of inviting teammates into the same workspace, nor of different permission levels (like "admin" versus "viewer"). This is a genuine, current limitation for team and enterprise use, discussed openly in Section 19.

Is access control enforced only in the visible screens, or does it hold up structurally? It holds up structurally. The actual page-level protection that stops a logged-out visitor from ever seeing a private screen happens on the server, before any private content is ever sent to the browser - not merely hidden by the visual interface after the fact, which would be a much weaker and easily bypassed form of protection. Every underlying data request is separately and independently checked as well, so even if someone tried to skip the visible screens entirely and talk to the backend directly, the same ownership checks still apply.

Overall assessment: no case was found, anywhere in this review, of a screen or a data request that relied only on the interface hiding something rather than the backend genuinely refusing it - which is the correct, secure way to build this kind of protection.

15. SECURITY THREAT MODEL

A structured look at realistic threats, who might cause them, and how well-defended each one currently is.

Threat	Who might cause it	What could happen	How well defended today	Severity	What would fix it
Someone impersonating another user	An outside attacker	Full account takeover	Every login is independently, cryptographically re-verified on every request	Low - well defended	-
One organization reading another's data	An outside attacker guessing or manipulating an address	Cross-organization data leak	Ownership is always independently re-checked against the real logged-in identity, everywhere	Low - well defended	Keep expanding automated testing for this as new features are added
A single very large or resource-heavy repository	Any logged-in user, even unintentionally	The scanning process could slow down or strain under an unusually large amount of data	No specific safeguard exists yet	Medium - real, open gap	Add a sensible size limit before processing begins
Excessive or repeated scan requests	Any logged-in user, even unintentionally	The system could be strained by being asked to do too much work too quickly	No specific safeguard exists yet	Medium - real, open gap	Add a reasonable limit on how often scans or questions can be requested
A compromised or malicious external code-hosting server	A sophisticated outside attacker	A theoretical risk during the download step itself, entirely dependent on the underlying download tooling being flawed, not on anything specific to this product	Not specifically isolated or sandboxed beyond using a private, temporary location	Medium - a real but low-likelihood residual risk	Run each scan in a fully isolated, disposable environment
A compromised AI provider	A sophisticated outside attacker	At most, a rejected or ignored bad response - see Section 11	Structurally limited by strict response-checking and citation verification	Low - well contained	-

Threat	Who might cause it	What could happen	How well defended today	Severity	What would fix it
Malware inside a scanned repository	Any user submitting a hostile repository	None to this system itself - code is only ever read, never run	Read-only by design	Low - well defended, by design	-

16. TECHNOLOGY CHOICES, AND WHY

Rather than listing every technical dependency, here is what actually matters about the choices made and why:

- **A modern, widely-used web framework** powers the interface - chosen because it supports genuinely secure, server-verified access control (Section 14), not just decoration hidden by the interface.
- **A modern, async-capable backend framework** handles all the real logic - chosen because scanning work naturally involves a lot of waiting (downloading, external service calls), and this style of framework handles that efficiently without needing a much heavier infrastructure setup.
- **A professionally managed, hosted database** stores everything - chosen deliberately over self-hosting a database, to get reliability and baseline security "for free" from a specialized provider rather than reinventing that expertise in-house.
- **A dedicated identity provider** handles all login and session security - a deliberate choice not to build authentication in-house, since that is a notoriously easy thing to get subtly wrong.
- **Genuine code-parsing tooling**, rather than simple keyword search, powers the core detection engine - chosen specifically because it avoids many of the false-positive traps that plain text search falls into (Section 4).
- **A well-known, purpose-built security-pattern-matching tool** supplements the code parser for more specific rules (like "flag this only if the key size is below a certain number").
- **Two independent AI providers**, used with an automatic fallback between them, power the assistant feature - chosen specifically so a temporary outage or usage limit on one provider doesn't take the whole feature down, which was genuinely tested and proven to work during this project's own development.
- **No new, heavy formatting tools were introduced for exporting documents** (like this very report) - existing lightweight capabilities were used instead, favoring simplicity over adding unnecessary dependencies.

What is explicitly not part of the current technology choices: a dedicated graph database, a dedicated background-job queue system, binary/container-scanning tooling, and any automated deployment pipeline - all discussed openly as future work in Section 20, not hidden.

17. HOW THE CORE ALGORITHMS WORK, CONCEPTUALLY

This section explains the *reasoning*, not the code, behind each major decision-making process in the product.

- **Finding cryptography in code** works by genuinely understanding a file's structure (not just its words) and comparing specific, meaningful parts of it - like an import statement or a function call - against a known list of cryptography-related patterns. This avoids a huge and well-documented class of mistakes plain text-searching tools make, like matching a coincidental substring inside an unrelated word.
- **Naming an algorithm** is a straightforward lookup against a curated reference dictionary of known algorithm names and their common spelling variations - deliberately not a guess, and explicitly refusing to invent a name when nothing matches.
- **Deciding if something is quantum- or classically-vulnerable** is, likewise, a lookup against two curated, maintained reference lists - not a judgment call made fresh each time.
- **Calculating risk** applies the "data lifetime plus migration time versus time remaining" reasoning described in Section 7, consistently and predictably, with no randomness involved - the same finding, evaluated twice, always produces the same result.

- **Recommending a replacement** works by matching a vulnerable algorithm's type and purpose against a curated table of known-good, currently-standardized modern replacements - again a lookup, not a generated or improvised suggestion.
- **Calculating the overall "Quantum Readiness" score** combines five separately measured aspects of an organization's posture (like how much of their code has actually been scanned, and how much progress has been made migrating known risks) into one weighted overall number - and, notably, is honest about a sixth aspect it cannot currently measure at all, showing it plainly as "not measured" rather than guessing a number for it.
- **The 3D relationship visualization** uses a simple, self-contained simulation purely for visual layout purposes - it decides where to place things on screen so they look clear and well-organized, but this simulation has no bearing on the actual facts or risk calculations shown; it is purely cosmetic organization of real data.
- **The AI assistant's underlying language model** is the only place in the whole system where a general-purpose AI model is actually used for reasoning - specifically for turning an already-correct, already-ranked set of real findings into natural, readable language, and never for deciding what those findings or their ranking actually are.

18. PERFORMANCE & SCALABILITY

A small project scans quickly, essentially without a noticeable wait.

A typical, moderate-sized project completes in roughly a minute or two, based on real scans observed during this project's own testing - the exact time depends mostly on how long the pattern-matching security tool takes to run, which tends to be the slowest single step.

A very large, enterprise-scale codebase would currently be handled less gracefully: every file is read fully before being analyzed, with no safeguard against an unusually large file; and the different scanning methods run one after another rather than genuinely in parallel with each other for a single project - so a very large project would take proportionally longer, without today's system actively working to speed that up.

Many repositories scanned across an entire large organization would run into a more fundamental limitation: today, all scan work for an organization is handled by one lightweight background process, without a dedicated task-queue system designed specifically for spreading heavy work across multiple workers - meaning true enterprise-scale concurrent scanning (thousands of repositories at once) is not something the current system is built to handle without further work.

Binary-heavy or compiled-code-heavy repositories are effectively invisible today, since binary analysis doesn't exist yet - this is a coverage gap, not a performance problem.

A real, notable improvement already made during this project: an earlier version of the system was found to be unnecessarily slow while processing very large scan results, because it was saving its findings to the database one at a time instead of in efficient batches - a real large scan that used to take over ten minutes now completes in a small fraction of that time, after the underlying process was corrected. This is mentioned specifically because it demonstrates the team's willingness to find and fix real performance problems through actual testing, not just claim good performance.

How this would realistically scale further: introducing a proper task-queue system so multiple scans can run genuinely in parallel across multiple workers, adding a sensible size limit per file, and adding basic rate limiting so no single user can unintentionally overwhelm the system - all reasonable, well-understood next steps rather than deep architectural rework.

19. CURRENT LIMITATIONS - STATED HONESTLY

Presented directly, with no attempt to soften them, because a security product's credibility depends on being honest about its own gaps.

- **No support for scanning private repositories.** Only publicly accessible repositories can be scanned today, since there is no login-based connection method built yet. This means the product currently cannot scan the exact code most organizations most want protected, without further work.
- **No detection yet for password-hashing or message-authentication algorithms.** This is a common and important category of real-world cryptography use that is not yet recognized at all.

- **No safeguard against unusually large files or repositories.** A large enough file could slow down or strain a scan; nothing currently protects against this.
- **No limit on how often scans or AI questions can be requested.** A user (even unintentionally) could request more work than the system is comfortable handling at once, with nothing currently stopping that.
- **The workspace settings screen currently shows several enterprise-style options (team members, single sign-on, security policies, integrations, and so on) that are not yet functional** - only one setting on that entire screen (the risk-calculation time assumption) is genuinely connected to real functionality today. This is worth being very direct about, since it would be easy for someone exploring the product to assume more is wired up there than actually is.
- **No support yet for multiple people collaborating inside one workspace**, nor for different permission levels between them - today it is strictly one person, one workspace.
- **No detection yet of cryptography inside compiled programs, container images, or cloud infrastructure configuration** - the product currently only understands source code, dependency manifests, and certificate files.
- **Two of the numbers feeding the risk calculation (how long a typical migration takes, and how many years remain before a capable quantum computer exists) are reasonable, named estimates, not measured facts** - one of the two is already something an organization can adjust themselves; the other is not yet.
- **The AI assistant's citations are checked, but its general prose is not independently fact-checked sentence by sentence** - a small, honestly-acknowledged gap in an otherwise carefully-guarded feature.
- **The "Dependency Graph" visualization only models a simple, two-level relationship (which project uses which algorithm)** - it is not the same as a full, multi-layer enterprise dependency map connecting applications, business systems, and data together, and shouldn't be described as more than it is.
- **No isolation exists yet around the repository-downloading step itself beyond it being temporary and private** - a genuinely sophisticated attack against the underlying download tooling is a low-likelihood but real residual risk that isn't specifically defended against today.
- **All scan work currently runs through one lightweight background process**, without a dedicated system built for spreading heavy work across multiple workers at true enterprise scale.
- **Every recommendation is currently shown with the same fixed confidence level, rather than one calculated individually per recommendation** - stated so the number is never mistaken for a genuinely computed statistic.
- **No formal, published accuracy benchmark exists yet** (a measured "this product correctly identifies X% of real cryptography and rarely mis-flags something that isn't") - real, specific examples of both kinds of mistakes were found and fixed during development, which is a healthy sign, but there is no single published number to point to yet.

20. FUTURE PHASES

Where things stand today: a genuinely working, end-to-end pipeline - discover cryptography, classify it correctly, calculate real quantum risk, recommend a real replacement, export a standards-aligned inventory, track migration through to a real, evidence-based verification, watch for policy violations, and ask an AI assistant honest, evidence-grounded questions about all of it.

The next, most valuable step: production hardening. Before adding new capabilities, the most responsible next investment would be closing the operational gaps already named directly in Section 19 - adding safeguards against oversized files, adding basic rate limiting, and isolating the repository-download step more strongly. None of this is glamorous, but it is exactly the kind of unglamorous work that separates a promising prototype from something ready for real, continuous, unsupervised use.

After that: genuine enterprise readiness. Real team collaboration with multiple people and permission levels per workspace, real single-sign-on support, connecting private repositories securely, an audit trail that can be exported into an organization's existing security monitoring tools, and moving from "scan when a person clicks a button" to continuous, automatic scanning triggered by new code changes.

After that: deeper cryptographic coverage. Recognizing password-hashing and message-authentication algorithms, understanding compiled programs and container images, understanding cloud infrastructure configuration, and - importantly - finally publishing a real, measured accuracy benchmark rather than relying on anecdotal examples of fixed mistakes.

After that: a genuinely deeper migration story. Letting an organization tune the migration-time assumption themselves the same way they can already tune the "years remaining" assumption, offering guided migration simulations ("what happens to our overall risk if we fix these five things first"), and using the product's own real, evidence-based verification history to eventually replace today's estimated migration-time assumptions with real, observed ones.

Finally: a more capable AI assistant, built on the same honest foundation already in place. The right way to grow the AI feature further is not to give it more autonomy or more trust, but to close the one honestly-acknowledged gap that remains today - independently checking the *general reasoning* of an answer, not just its specific citations - while keeping the same non-negotiable principle the product already holds itself to: the AI explains real evidence, it never becomes the source of it.

21. STANDARDS & COMPLIANCE

It matters to be precise about the difference between a product's approach being **aligned with** an established standard, and that product having been **formally certified compliant** with it - these are genuinely different things, and this document is careful not to blur them.

Area	Honest status
Recommended post-quantum replacement algorithms	Aligned with the current, official, formally standardized algorithms in this space
Guidance on which older algorithms are already considered too weak to use	Aligned with widely recognized, published industry guidance
The exported inventory document's format	Aligned with a recognized industry standard's structure, and self-checked against its core rules - not independently, externally certified
General security-testing best practices	Not yet formally evaluated against a published checklist
Formal organizational security certifications (the kind of certification an entire company earns, not a single tool)	Not applicable to a codebase in isolation, and not claimed
Data-privacy regulation (the kind that governs how personal data must be handled)	Not yet formally evaluated - for example, there is currently no built-in way for a user to request their data be fully deleted

The honest rule this document follows: nowhere does it claim formal "compliance" or "certification" for anything that has not actually been independently verified by an outside authority - it only ever claims alignment with the spirit and substance of a standard, which is a real and meaningful thing, just not the same thing as certification.

22. JUDGE ATTACK MODE

Fifty likely questions, grouped by topic, each with a genuine, defensible, plain-language answer and - just as importantly - the wrong, overreaching answer to avoid.

Product

- What does this product actually do, in one sentence?** It finds real cryptography in an organization's code, tells them honestly how at-risk it is against quantum computing, and recommends a real, standards-based fix - with an AI assistant layered on top purely to explain that data in plain language. *Avoid saying:* "the AI finds the vulnerabilities."
- Is this a finished product or just a prototype?** A genuinely working prototype with real, tested, end-to-end functionality - not a mockup - but openly not yet hardened for unsupervised, large-scale production use.
- What's the single most important number this product produces?** An overall "Quantum Readiness" score - but deliberately always shown broken down into its real components, and honest about the one piece of it that currently

cannot be measured at all, rather than ever presented as a mysterious single number.

Architecture

4. **Why split the interface and the backend logic into two separate pieces instead of one?** So the interface can be genuinely secured on its own terms, and the more specialized code-analysis and cryptography tooling can live in a backend built specifically for that kind of work.
5. **Why no dedicated task-queue system yet?** A simpler built-in mechanism was sufficient at this stage; the groundwork for upgrading to a proper task queue later is already anticipated and openly planned as a next step, not something being hidden.
6. **Where's the knowledge graph?** Addressed directly in Section 10 - there isn't a full one yet; what exists today is a real, honestly-scoped, simpler relationship view.

Cryptography

7. **How do you actually detect RSA in a piece of code?** By genuinely understanding the code's structure and recognizing specific, known patterns of RSA being imported or called - not by simply searching for the word "RSA" as text.
8. **What's your false-positive rate?** Not yet a single published number - but several real, specific examples of false positives were found and fixed during development, which is a genuinely good sign of active testing, even without one final aggregate statistic to quote yet.
9. **Can you detect a completely custom, homegrown encryption algorithm someone wrote themselves?** Only if it happens to be built in a way that still resembles a recognizable pattern (for example, following common naming conventions) - a truly novel implementation with no recognizable structure would not be detected. *Avoid claiming:* universal detection of any cryptography whatsoever.

Quantum Computing

10. **Is RSA safe if you just use a bigger key?** No - a larger key size does not help against a quantum computer at all, because the entire underlying mathematical approach is broken, not merely weakened. The product models this correctly.
11. **Is AES-256 quantum-safe?** For all practical purposes, yes - a quantum computer weakens it, but it remains strong enough to be considered acceptable.
12. **When exactly will a quantum computer capable of breaking today's cryptography exist?** Genuinely unknown - nobody in the field can say for certain. The product's own working assumption about this is openly labeled as an editable estimate, not a claimed fact. *Avoid ever stating a specific year with confidence.*
13. **What is "harvest now, decrypt later"?** The very real risk that an adversary captures and stores encrypted data today, intending to decrypt it later once quantum computing matures enough - which is exactly what the product's core risk calculation is built around.

AI

14. **Does the AI ever make things up?** Its general phrasing could, in principle, be imprecise, but every specific factual claim it makes is independently checked against real data before ever being shown, and anything that doesn't check out is quietly removed rather than displayed.
15. **What happens if your AI provider goes down?** The system automatically tries a second, independent provider - genuinely tested and proven to work during this project's own development.
16. **Can the AI change any of my real data?** No - it can only read a summary of existing data and hold its own separate conversation; it has no ability to alter a risk score, a recommendation, or a migration status.
17. **Is my source code sent to an outside AI company?** No - only a summary of where a finding is and what kind of risk it represents is ever sent, never the actual matching code.
18. **Why not just let AI do the detection instead of building all these separate detection methods?** Because a detection tool's most important property is finding the exact same things every single time, reliably and completely -

which is exactly what predictable, rule-based methods do well, and what a general-purpose language model is not well-suited to guarantee.

Security

19. **What happens if I scan someone else's public repository without permission?** The system has no way to verify repository ownership - it simply scans whatever public address it's given, the same way anyone's own browser could visit that same public address.
20. **Can one user ever see another organization's data?** No - every single piece of data is tied to one workspace, and every request is independently re-checked against the real, verified identity of whoever is asking.
21. **What stops a malicious repository from attacking your system?** Code inside a repository is only ever read, never executed - so there is no direct path for hostile code to run inside the product itself.
22. **How do you prevent someone from overwhelming your system with huge scans?** Honestly - not fully prevented yet; this is a real, acknowledged, currently open gap (Section 15, Section 19). *Avoid claiming this is already solved.*
23. **Is your login system something you built yourselves?** No - login and session security is handled by a dedicated, specialized identity provider chosen specifically because that is a deep specialty best left to a provider whose whole business is getting it right.

Privacy

24. **Do you store people's personal information?** Not intentionally or specifically - only whatever might incidentally sit on the same line as an actual cryptography finding, and even then, only that specific line, never a whole file.
25. **Can a user ask to have their data deleted?** Not through a built-in feature yet - a genuine, open gap relevant to data-privacy regulation, stated directly rather than avoided.
26. **What actually leaves your system and goes to an outside AI provider?** Only finding summaries relevant to a specific question - never actual source code.

Accuracy

27. **How do you know your risk scores are actually correct?** The underlying reasoning method is sound and consistently applied - but two of its input assumptions (typical migration time, and years remaining before a capable quantum computer exists) are honest, named estimates, not measured facts, and one of them is already adjustable by the organization for exactly that reason.
28. **How do you know your algorithm classification is correct?** It comes from a straightforward, auditable lookup against a curated, maintained reference list - reliable for anything on that list, and honestly unable to name anything that isn't.

Data

29. **What happens to old findings after a problem is actually fixed?** Nothing is ever deleted - old findings remain as a permanent historical record, and the system separately, freshly re-confirms through a real re-scan whether the problem is actually gone before marking anything as verified.
30. **How do you avoid showing the same finding as ten confusing duplicate rows?** The display groups repeated occurrences of the same finding together and shows how many times it's been seen, while still keeping every individual underlying occurrence intact behind the scenes.

Scalability

31. **Could this handle scanning ten thousand repositories for a large enterprise today?** Not without further work - openly acknowledged as a current limitation (Section 18), not something to overstate.
32. **What's the single biggest thing limiting how much this can scale right now?** The lack of a dedicated system for spreading scan work across multiple workers at once - today, it's handled by one lightweight process.

Enterprise

33. **Do you support single sign-on?** Not yet - it's shown as a planned item in the settings area, but it isn't functional today. *Avoid implying it works because it's visible in the interface.*
34. **Can multiple people from the same team share one workspace?** Not yet - today it is strictly one person per workspace.
35. **Can this connect directly to GitHub or GitLab for automatic, continuous scanning?** Not yet - scanning is currently triggered manually, on demand.

Business

36. **What's genuinely different about this compared to other tools in this space?** The combination of strictly evidence-first, predictable detection with an AI layer that is architecturally forbidden from inventing findings - many competing "AI security" tools lead with AI-driven detection, which carries a meaningfully higher risk of confidently-wrong answers.
37. **What's the weakest part of the product today?** Honestly, enterprise readiness - no private-repository support, no team/role system, no single sign-on yet. Stated directly, not hidden.

Standards

38. **Are you certified compliant with post-quantum cryptography standards?** The recommendations are aligned with the current, official, formally standardized replacement algorithms - "aligned with," not a formal certification, and that distinction is intentional (Section 21).
39. **Do you follow recognized government/defense-grade cryptography guidance?** The reasoning and recommendations are aligned with it and reference it directly - again, alignment, not a formal audit or certification.

Future roadmap

40. **What's the very next thing you'd build?** The two smallest, highest-value fixes named directly in this document's own limitations: a safeguard against oversized files, and basic rate limiting - deliberately chosen because they're small, well-understood, and meaningfully reduce real risk.
41. **Would you build binary or container scanning next?** It's intentionally further down the roadmap than that - the team made a deliberate, documented choice to prioritize getting the core risk, verification, and honesty features right first, rather than chasing broader coverage before the foundation was solid.

Additional cross-cutting questions

42. **What's the single most impressive engineering decision in this product?** Its consistent discipline around never fabricating data - showing an unmeasured number as genuinely unmeasured rather than guessed, never inventing a named algorithm out of an unrecognized finding, and never letting a "what if" experiment quietly overwrite real results. This shows up repeatedly, not just once.
43. **What's the single riskiest engineering decision today?** Allowing arbitrary repository downloads with no size limit and no rate limiting yet - a real, stated, currently open gap.
44. **How many real problems have you actually found and fixed in your own system?** A genuine, dated list exists and is openly referenced throughout this project's own history - including a real timestamp-handling bug, a real performance problem, a case where an experimental "what if" feature was silently overwriting real data before being fixed, and the AI-provider usage-limit issue fixed this same week. Being able to point to this list honestly is itself a sign of a real, actively operated and debugged system, not a static demo.
45. **How thoroughly is this actually tested?** A substantial, passing set of automated checks exists covering the backend's real logic and real database behavior - not just surface-level demonstration testing.
46. **Could a competitor copy this in a week?** The individual detection rules and recommendation lists, probably fairly quickly. The deeper discipline - never fabricating data, provably reversible "what if" experiments, an AI layer that's architecturally forced to stay honest - would take meaningfully longer to replicate correctly, because it has to be built

into the foundation, not added afterward.

47. **What's the honest overall maturity level of this product?** A solid, genuinely working core (find it, classify it, score it, recommend a fix, prove it's actually fixed) with clearly, deliberately labeled gaps everywhere else - nothing found during this review was overstated beyond what's actually built.

48. **If you only had one more week, what would you build?** The file-size safeguard and basic rate limiting named throughout this document - small, well-understood, and the highest-leverage fixes among everything currently open.

49. **What would you tell a skeptical enterprise buyer today?** That the core cryptographic discovery and risk-scoring engine is genuinely solid and already trustworthy, but that private-repository support and team/enterprise features are honestly not ready yet - and that this document exists specifically so nobody has to take that on faith.

50. **What's the one thing you're proudest of in this build?** The Verification feature - refusing to let "we fixed it" be a claim anyone can simply assert, and instead requiring the system to prove it with a fresh, real re-scan before ever marking something resolved.

23. HARD QUESTIONS - DIRECT ANSWERS

1. **Why can't existing code-scanning security tools already solve this?** Most existing tools are built to find leaked secrets or general security bugs - they have no concept of "this specific algorithm is vulnerable to a quantum computer" or how to calculate that kind of risk over time.

2. **Why isn't this just a security-pattern-matching tool by itself?** That kind of tool is only one of four detection methods this product uses, and on its own has no concept of a unified inventory, a risk calculation, a standardized export, or a migration-tracking workflow.

3. **Why isn't this just a standard software inventory list?** A standard inventory tells you which external packages a project uses - it has no idea that a specific line of code, three files deep, is actually calling a weak algorithm with a small key, and it has no concept of quantum risk at all.

4. **Why do you need a specialized cryptography inventory format instead of a general one?** Because "this is a cryptographic algorithm, here's its key size and quantum status" isn't information a general software inventory format is built to represent at all - a specialized, purpose-built format is what actually captures that.

5. **Do you genuinely need a full knowledge graph?** Honestly, not for what's demonstrated today - the simpler, real relationship views already in the product cover today's real use cases. A full, multi-layer graph would matter more at a much larger, more complex enterprise scale, and is realistically future work rather than something the product is missing today.

6. **Why use AI at all, if it's not doing the actual detection?** Because turning dozens of individually correct findings into one clear, readable answer to "what should I fix first" is genuinely something natural language is better at than a raw table - but the *correctness* of that answer was already guaranteed before the AI ever got involved.

7. **Why shouldn't AI be trusted to do the actual detection?** Because detection needs to be exhaustive and perfectly repeatable - the same code should always produce the same findings - and a language model, by its nature, isn't built to guarantee either of those things.

8. **How do you actually stop the AI from making things up?** Not by asking it nicely - by independently checking every specific fact it claims against the real database afterward, and quietly removing anything that doesn't check out, rather than trusting the AI's word.

9. **How do you know your risk score is actually right?** The reasoning method behind it is sound and industry-recognized; two of the numbers feeding into it are honest, named estimates rather than measured facts, and the system is upfront about that rather than presenting them as certainties.

10. **How do you know your algorithm naming is actually right?** It comes from a maintained, curated reference list, matched exactly - reliable for anything on that list, and it deliberately refuses to guess at anything that isn't, rather than risk being confidently wrong.

11. **What happens with a completely custom cryptographic algorithm someone invented themselves?** It would likely go undetected, unless it happens to resemble a recognizable pattern - a real, honest limitation.

12. **What happens with a compiled program instead of source code?** Currently invisible - binary analysis doesn't exist yet in this product.

13. **What happens with cryptography that's generated dynamically at runtime, rather than written plainly in the code?** Not detected - all analysis happens on the code as written, not on what it might do while actually running.

14. **What happens if the repository someone wants scanned is private?** The scan simply cannot proceed - there's currently no way to authenticate into a private repository at all.
15. **Is source code genuinely sent to an AI company, or is that just a policy promise?** It's a structural guarantee, not just a policy - the actual code text is deliberately excluded from what gets built into any AI request in the first place, not merely instructed not to be included.
16. **What's the worst that could happen if your AI provider itself were compromised?** At most, a response that gets rejected for not matching the expected structure, or specific claims that get silently filtered out for not matching real data - not a breach of real, stored data.
17. **What if the scanner misses cryptography that's hidden behind a layer of custom abstraction in someone's code?** It would be missed, honestly - detection works at the level of the actual recognizable call or import, not through layers of custom indirection, and this is a stated limitation, not a hidden one.
18. **What's your false-positive rate?** Not yet a single published number - genuine examples were found and fixed during development, which is meaningful evidence of real testing, though not yet a formal benchmark.
19. **What's your false-negative rate?** Same honest answer - qualitatively, known gaps exist (an entire category of common algorithms isn't detected yet, for instance), but nothing has been formally measured and published yet.
20. **How do you actually validate detection accuracy?** Through a dedicated set of known, verified real-world examples used specifically to test the system against - functional testing, not yet a full published statistical benchmark.
21. **How do you handle keeping different organizations' data separate?** Every single piece of data is permanently tied to the one workspace it belongs to, and ownership is independently re-verified on every single request.
22. **What specifically prevents one organization's data from leaking into another's?** The same answer as above, applied consistently and without exception across every part of the system that was reviewed.
23. **How do you handle private encryption keys found during a scan?** They are never extracted or stored - only the public, non-secret metadata about a certificate (its algorithm, key size, and validity) is ever kept.
24. **What happens if a scanned repository contains genuinely malicious code?** It's read, never executed - so hostile code inside a repository has no way to actually run inside this product.
25. **Could the scanning process itself become a way to attack the system?** The most realistic version of that risk lives in the underlying download tooling itself being exploited, not in anything specific to how this product processes what it downloads - and it's a real, if low-likelihood, residual risk that isn't specifically defended against yet.
26. **How would this realistically scale to ten thousand repositories?** Not without further investment in the areas already named honestly in Section 18 and Section 20 - a direct, non-evasive "not yet, and here's exactly what it would take" is the right answer.
27. **What genuinely makes this different from other tools that already exist in this space?** The combination, taken together, of evidence-first detection, an honest and adjustable risk model, a real proof-based verification step, and an AI layer that is structurally forbidden from inventing findings - not any single one of these alone.
28. **What's the strongest, most defensible technical strength of this product?** Its discipline around never fabricating data - shown consistently and repeatedly across the whole product, not as a one-off feature.
29. **What's the weakest part of the product, stated plainly?** Enterprise readiness - private-repository support, team/role management, and single sign-on are all currently missing.
30. **What would genuinely be built next, if this continued?** The two smallest, most direct safety fixes already named throughout this document - a safeguard against oversized files, and basic rate limiting - chosen deliberately because they are small, well-understood, and meaningfully reduce real, currently-open risk.

24. COMPETITIVE POSITION

What matters	This product	What most existing tools in nearby categories do	Where this product has a genuine edge	Where it honestly has a gap today
Finding cryptography in real code	Genuine, structural code understanding across several languages, purpose-built specifically for cryptography	General-purpose security-scanning tools usually look for broader categories of bugs, not a dedicated cryptography inventory	A cryptography-specific vocabulary and classification system built for exactly this problem	Narrower language coverage than a long-established, general-purpose scanning tool would have
A standardized cryptography inventory export	Real, genuinely generated, in a recognized industry format	Most general software-inventory tools don't produce this kind of document at all yet - it's a newer, emerging idea in the industry	Native, working support for one of the newest and most specific standards in this exact space	Self-checked internally, not yet independently certified by an outside authority
Quantum risk scoring	Real, concrete, auditable, and explainable	Most general security or software-inventory tools have no concept of quantum risk at all; a newer, smaller category of dedicated tools is starting to emerge here too	A clear, explainable calculation rather than an unexplained black-box score	Two of the calculation's underlying assumptions are estimates, not yet fully organization-tunable
An AI assistant	Real, but deliberately kept read-only and evidence-checked	Many competing "AI-powered" security products lead with AI actually performing detection, which carries meaningfully more risk of confidently-wrong answers	The AI is provably unable to invent or alter findings - a real, structural trust advantage	The general prose isn't independently fact-checked the same way individual citations are
Team and enterprise features	Basic, single-user-per-workspace only	Mature, established enterprise security platforms typically offer full team management, permissions, and single sign-on	-	A real, current, and openly acknowledged gap
Coverage beyond source code (compiled programs, containers, cloud)	Not yet present	Established cloud-security platforms typically already cover this	-	A real, currently deferred gap, by deliberate choice rather than oversight

The honest, factual positioning: this product is not claiming to outperform large, established, heavily-funded security platforms on sheer breadth of coverage. Its genuine, defensible claim is narrower and real: a cryptography-specific, evidence-first tool that is unusually disciplined about never overstating what it actually knows - a real and currently underserved niche, not an exaggerated one.

25. THE 5-MINUTE DEMO STORY

Time	What to show	What to say	What to emphasize	What to avoid saying
0:00-0:30	The problem framing	"Every organization has cryptography buried in code they can't fully inventory - and quantum computers will eventually break the most common kind of it, not someday in the abstract, but within a timeframe security teams genuinely need to plan for today."	The "harvest now, decrypt later" idea, explained simply	Don't state a specific year with confidence
0:30-1:00	A simple picture of the system	"An interface people use, a backend that does all the real thinking, a database that remembers everything permanently, and an AI layer that only ever explains - never decides."	That the AI is explicitly read-only	Don't claim a full knowledge-graph system exists
1:00-2:00	Live: connect a real repository and run a scan	"Watch it read the actual code and produce real evidence - not a guess, an actual file and an actual line."	Point at one specific, real finding	Don't claim binary or container scanning
2:00-3:00	The inventory export and the relationship view	"A standardized document, ready to hand to an auditor. And this view shows, using real data, which projects actually use which algorithms."	Say plainly that this is computed from real evidence, not hand-arranged	Don't call the relationship view a full graph database
3:00-4:00	The risk calculation and the Migration Planner, ending on Verification	"This is Mosca's reasoning, applied for real. And here's the part I'm proudest of: dragging a card to 'done' doesn't actually mark it done - only a fresh, real re-scan confirming the old algorithm is genuinely gone does that."	Spend real time here - this is the single strongest, most defensible claim in the whole demo	Don't imply the risk model's underlying time assumptions are measured facts rather than adjustable estimates
4:00-4:40	The AI Analyst	Ask it a real question live, and show its answer linking back to real, clickable evidence.	"Every specific thing it claims is checked against real data before you ever see it."	Don't claim the AI performs any detection or writes any data
4:40-5:00	Honesty about security and what's next	"No source code ever leaves this system for the AI. Private-repository support and basic rate limiting are the honest, direct next steps - not hidden gaps, just the next things on the list."	State the real, current gaps yourself, in your own words	Don't claim single sign-on or team management already exist

26. FINAL "KNOW YOUR PRODUCT" REFERENCE

In one sentence: This product finds real cryptography in an organization's code, explains honestly how at-risk it is against quantum computing, and recommends a real, standards-based fix - with an AI assistant that explains this evidence but can never invent or alter it.

In 30 seconds: Point it at a public code repository. It downloads a temporary copy, reads the code using several independent and predictable methods, finds real cryptography, correctly names each algorithm, calculates a genuine risk level using well-established reasoning about quantum computing timelines, recommends a modern replacement, and produces a standardized, exportable inventory document. A team can track fixing each finding through a simple planning board - and, critically, nothing is ever marked "done" on the system's own authority until a fresh re-scan actually proves the old problem is gone. An AI assistant lets anyone ask plain questions about all of this, with every specific claim it makes independently checked against real data before being shown.

In two minutes: Walk through the architecture (Section 1), the scanning journey (Section 3), the risk reasoning (Section 7), and the AI's evidence chain (Section 11) - in that order, out loud, in your own words.

- **Complete architecture** - Section 1
- **Complete data flow** - Section 3
- **Every tab explained** - Section 2
- **Every algorithm covered** - Section 6
- **Technology choices and why** - Section 16
- **Security model** - Sections 13-15
- **AI model** - Section 11
- **Quantum risk model** - Sections 7-8
- **The inventory export** - Section 9
- **The knowledge-graph honesty check** - Section 10
- **Current limitations** - Section 19
- **Future roadmap** - Section 20
- **Top judge questions and answers** - Sections 22-23

Ten things to never claim

1. That the AI detects vulnerabilities - it never does; detection is entirely separate and predictable.
2. That the inventory export is formally, independently certified - it is self-checked and aligned with the standard, not externally certified.
3. That team management, single sign-on, or role-based permissions exist - none of them do yet.
4. That there's certainty about when a quantum computer capable of breaking today's cryptography will exist - that number is an editable assumption, not a prediction.
5. That private repositories can be scanned - only public ones can, today.
6. That a full knowledge graph exists - it doesn't; be precise about what the relationship views actually show instead.
7. That confidence scores are individually, statistically calculated per finding - they are fixed values tied to which detection method found something.
8. That every kind of cryptography is detected - an entire common category (password-hashing and message-authentication algorithms) isn't recognized yet, and neither is anything hidden in a compiled program.
9. That scans are rate-limited or sandboxed against abuse - neither protection exists yet; say so plainly if asked.
10. That the recommendation confidence number reflects a real statistical calculation - it's a fixed value applied to every recommendation, not a computed metric.

27. TECHNOLOGY STACK, IN DETAIL

This section names every real technology behind ECDAT and explains, in plain language, what it actually is and why this particular product needs it - including the two tools most worth understanding deeply: Tree-sitter and Semgrep, the two engines that do the actual work of finding cryptography in code.

27.1 The interface layer

Next.js / React - the framework the entire web interface is built on. In plain terms, React is a way of building a web page out of small, reusable, self-contained pieces (a button, a table, a card) that each know how to redraw themselves the instant the data behind them changes - which is why the dashboard updates live as new scan results come in, instead of needing a manual page refresh. Next.js is a larger toolkit built around React that adds, among other things, the ability to check "is this person actually logged in" on the server, before a single byte of a private page is ever sent to the browser - which is exactly the mechanism that keeps the private, authenticated part of the product genuinely secure rather than just visually hidden.

TypeScript - a stricter version of the JavaScript programming language that catches a large class of simple mistakes (like accidentally treating a number as if it were text) before the product is even run, rather than only discovering them later when a real user hits the bug.

Tailwind CSS - a styling toolkit used to build the visual look of the interface quickly and consistently, without writing custom styling rules from scratch for every single element.

Framer Motion - a library specifically for animation - the smooth fades, slides, and transitions seen throughout the interface (like the workspace setup sequence, or a panel sliding into view) come from this.

React Three Fiber, and the underlying "Three.js" graphics engine - the technology behind the rotatable 3D dependency-graph view. Three.js is a general-purpose toolkit for drawing genuine 3D graphics inside a web browser (the same category of technology behind browser-based 3D games); React Three Fiber is simply a bridge that lets that 3D graphics engine be controlled using the same component-based style as the rest of the interface, so the graph can react live to real data the same way any other part of the page does.

27.2 The backend / logic layer

FastAPI - the framework the entire backend runs on. It is specifically built for the kind of workload this product has a lot of: many requests that spend most of their time *waiting* (waiting for a database reply, waiting for an external download, waiting for an AI provider to respond) rather than doing heavy calculation - and it's built to handle a large number of those waiting requests efficiently, at the same time, without needing separate heavyweight infrastructure for it.

SQLAlchemy, and Alembic alongside it - the toolkit the backend uses to talk to the database, and to safely evolve the database's own structure over time as the product grows. Rather than writing raw database commands by hand throughout the codebase, this toolkit lets the backend describe data as familiar programming objects, while still ultimately talking to a real, standard database underneath. Alembic specifically keeps a versioned, ordered history of every structural change ever made to the database - so the database's shape can always be reliably reproduced or rolled forward, the same way a document's version history lets you see exactly what changed and when.

PostgreSQL, hosted via Supabase - the actual database itself. PostgreSQL is one of the most established, trusted, general-purpose databases in the industry; Supabase is a hosting service that runs and manages a PostgreSQL database professionally, so the product benefits from a well-run, properly maintained database without the team needing to operate that infrastructure themselves from scratch.

27.3 Identity and access

Clerk - the dedicated identity provider used for login, sign-up, and session security. Rather than the product inventing its own login system (a notoriously easy thing to get subtly wrong, with serious consequences if it's wrong), this responsibility is handed to a provider that specializes in nothing else - and, critically, the backend independently re-checks the cryptographic proof this provider issues on every single request, rather than simply trusting whatever the browser claims.

27.4 The detection engine - explained properly

This is the part of the technology stack worth understanding in real depth, because it's the actual core of what the product does.

Tree-sitter - what it is and how it really works. Tree-sitter is a tool that reads source code the way a compiler's very first stage does: instead of treating a file as a flat block of text to search through, it genuinely understands the *grammar* of a programming language and builds a structured, tree-shaped representation of the code - technically called a "syntax tree" - where every meaningful piece (an import statement, a function call, an argument) becomes its own distinct, identifiable element in that tree, correctly nested inside whatever it actually belongs to. Because of this, the detection engine can ask a precise question like "is this specifically an import statement, and does it specifically name a known cryptography library" - rather than a much cruder question like "does this file merely contain the *word* 'crypto' somewhere." This is exactly what avoids a well-known class of mistake in simpler tools: a plain text search would wrongly flag a word like "modes" just because it happens to contain the letters "des" inside it, purely by coincidence - a real structural parser never makes that mistake, because it understands that those letters aren't actually a separate, meaningful piece of the code at all. Tree-sitter is also fast enough to parse code incrementally and reliably across many different programming languages, which is why it was chosen as the foundation of the detection engine specifically, rather than a simpler pattern-matching approach.

Semgrep - what it is and how it really works, and how it's different from Tree-sitter. Semgrep is a tool for writing *structural* search rules that look almost exactly like a small snippet of real code - but with special placeholders that can stand in for "any value here," and with the ability to attach extra conditions to those placeholders, like "and this value must be smaller than a specific number." This is a meaningfully different and complementary capability to Tree-sitter's own pattern matching: it lets the product express rules like "flag this specifically if a key is being generated with a size below a certain safe threshold" - a nuanced, comparison-based condition that goes beyond simply recognizing "this is a known import or function call." In plain terms: Tree-sitter is what lets the system correctly *recognize the shape* of code; Semgrep is what lets the system additionally *reason about specific values and conditions* within code that has already been recognized as relevant. Both tools are used together, on the same code, because each is genuinely better suited to a different part of the problem.

Reading dependency lists (manifest scanning) - many programming ecosystems require a project to explicitly declare, in a simple list, every external library it depends on. The detection engine reads these lists directly and checks each declared library's name against a maintained list of known cryptography-related libraries - a fast, highly reliable way to catch cryptography that's being used *through* a well-known library, rather than written by hand.

Reading digital certificates - for any certificate file physically present in a scanned repository, the engine performs genuine, standards-based parsing of that certificate's real structure, extracting real information from it (which algorithm and key size were used to sign it, who it belongs to, and when it expires) - not a guess or an approximation, but an accurate reading of the certificate's actual, real contents.

GitPython - the tool used to actually perform the "clone a repository" step - it handles the mechanics of talking to a code-hosting server and pulling down a working copy of a project, the same underlying technology that a developer's own everyday tools use.

27.5 The AI layer

What a "large language model" actually is, in plain terms - a large language model is a system trained on enormous amounts of text that has learned the statistical patterns of language well enough to generate fluent, coherent, contextually appropriate text in response to a prompt. It does not "know facts" the way a database does; it generates plausible-sounding language based on patterns - which is exactly why this product is careful to treat it as a narrator over real data, never as the source of the data itself (explained fully in Section 11).

Google Gemini and Groq - the two AI providers used, one as the default and one as an automatic backup. Gemini is Google's own large language model, accessed through their official service. Groq is a separate company offering access to large language models with an emphasis on very fast response speed. Using two independent providers, with automatic fallback between them, means a temporary problem with one provider doesn't take the assistant feature down entirely - a real scenario that was actually tested during this project's own development, when the primary provider hit a temporary usage limit and the backup handled the request successfully instead.

27.6 The trust and data-integrity libraries - easy to miss, genuinely important

Two more real, specific libraries sit underneath several features already described elsewhere in this document, and are worth naming directly rather than only describing what they enable.

The cryptography library. This is a real, industry-standard cryptographic toolkit - and, notably, it is the one genuinely "cryptographic" piece of software this product itself depends on internally, separate from the cryptography it goes out and

discovers in other people's code. It does two distinct jobs here: first, it is the actual engine performing real, standards-based reading of any digital certificate found during a scan (Section 4) - genuinely parsing the certificate's true structure to extract its real algorithm, key size, and validity information, not approximating it. Second, it works underneath the login-verification process described next, providing the underlying mathematical operations needed to check a cryptographic signature.

The JWT-verification engine (python-jose). This is the specific piece of software responsible for the security claim made throughout this document that login is "genuinely, independently re-verified" rather than simply trusted. When a person logs in, the identity provider issues them a signed token as proof. This library is what actually performs the real cryptographic check confirming that signature is authentic and was genuinely issued by the identity provider - not merely checking that the token *looks* correctly formatted. This is precisely the mechanism that stops someone from forging a fake login token and being believed.

The data-contract and validation engine (Pydantic). This is the library responsible for strictly defining the exact expected shape of data throughout the backend - and it plays a specific, important role already referenced in Section 11: when the AI assistant responds, this is the actual mechanism that checks whether the response genuinely matches the expected structure (an answer, a confidence level, a list of real citations) before it's ever trusted or shown to a user. If the AI's response doesn't match that expected shape, this is the engine that catches it and causes the honest "malformed response, rejected" behavior described earlier - rather than the system simply hoping the AI's output is well-formed.

27.7 ECDAT's own custom-built engines

Beyond the third-party tools above, the product also contains several genuinely custom-built decision engines of its own - each one a small, focused, independently understandable rulebook responsible for exactly one kind of decision. None of these are borrowed from an outside library; each was purpose-built for this product. They are named individually here, gathered in one place, because each is described in more depth elsewhere in this document and deserves to be recognized as its own distinct engine, the same way Tree-sitter and Simgrep are.

- **The Normalization Engine** - takes raw, differently-worded findings and matches each one to a single, consistent, correctly-named algorithm, refusing to guess when nothing matches (Section 5).
- **The Risk Engine** - takes a classified finding and calculates its real, explainable risk level using the quantum-risk reasoning described in Section 7, consistently and predictably every time.
- **The Recommendation Engine** - matches a vulnerable finding against a curated table of currently-standardized, modern replacement algorithms and produces a concrete, explained recommendation (Section 6).
- **The Readiness Engine** - combines several separately measured aspects of an organization's overall posture into the single "Quantum Readiness" score shown on the main dashboard, while honestly reporting the one aspect it cannot yet measure as genuinely unmeasured, rather than guessing (Section 2, Section 17).
- **The Policy Engine** - continuously checks every finding against a small set of baseline security rules (for example, "this specific algorithm should never be used at all"), and is what powers the Compliance screen's live list of violations and alerts (Section 2).
- **The Verification Engine** - the mechanism behind the product's proof-based migration workflow: it re-runs a genuinely real scan of a project and checks, using fresh evidence, whether a previously-flagged algorithm is actually gone - the engine specifically responsible for the distinction between "a person claims this is fixed" and "the system has proven this is fixed" (Section 2).
- **The CBOM Generator** - builds the standardized, exportable cryptographic inventory document described in Section 9, and separately checks its own output against the core rules of the standard it follows before producing it.
- **The Report Generator** - assembles the same underlying real data (risk levels, policy violations, migration progress) into a readable, shareable summary document for two different audiences: a short executive-level overview, and a fuller, more detailed technical version.

27.8 The graph visualization - explained as an algorithm

Force-directed layout - what it is and how it actually works. This is a genuinely classic and well-known technique for automatically arranging a set of connected items (like the "which project uses which algorithm" relationships shown in the Dependency Graph screen) into a visually clear picture, without a person having to manually decide where each item should sit. The idea is borrowed directly from physics: every item gently pushes every other item away from itself, the same way two magnets with matching poles repel each other - which naturally spreads everything out and prevents items

from overlapping. At the same time, any two items that are actually *connected* to each other are pulled together, the same way a spring pulls its two ends back toward each other - which naturally keeps genuinely related items close together on screen. A gentle overall pull toward the center keeps the whole picture from drifting apart entirely. These two opposing forces are recalculated many times per second until the layout settles into a natural, readable arrangement - one where closely related items cluster together and unrelated items spread apart, purely as a side effect of the physics, without anyone having to draw it by hand. It's worth being clear that this technique only decides *where things are drawn on screen* - it has no bearing whatsoever on the real underlying facts or risk calculations being displayed; it is purely a visual organizing technique layered on top of already-real data.

28. HOW EACH CRYPTOGRAPHIC ALGORITHM ACTUALLY WORKS

Section 6 listed which algorithms the product recognizes. This section explains, in plain language, what each of these algorithms actually *is*, what real-world problem it solves, and why some of them are fundamentally broken by quantum computing while others are merely weakened.

Public-key (asymmetric) cryptography, as a concept

Almost every algorithm in this category solves the same basic problem: how can two people who have never met, and share no prior secret, communicate securely - or how can one person "sign" something in a way nobody else could have produced, but anybody can verify? The answer, in every case, is a **mathematical pair of keys** - one key that can be shared publicly with anyone, and a second, mathematically related key that must be kept completely private. Something locked with the public key can only be unlocked with the matching private key, and something signed with the private key can be verified by anyone using the public key - but, critically, knowing the public key should never make it practical to work out the private key. Every algorithm in this family is built around a different underlying mathematical problem that is believed to be extremely hard to reverse using an ordinary computer.

- **RSA** - the oldest and most widely deployed public-key algorithm still in everyday use. Its underlying hard problem is factoring a very large number into its two secret prime-number building blocks - trivial to multiply two primes together, but (for a sufficiently large number, using an ordinary computer) astronomically difficult to reverse. RSA is used both for securely establishing a shared secret between two parties and for digital signatures. **Why it's quantum-broken, not just weakened:** a quantum computer running a specific, well-known quantum algorithm can solve exactly this kind of factoring problem efficiently - meaning a large enough quantum computer wouldn't just make RSA harder to crack, it would make it trivial, regardless of how large the key is made.
- **Diffie-Hellman (and its elliptic-curve variant, ECDH)** - a way for two parties to jointly agree on a shared secret over an insecure channel, without ever directly transmitting that secret itself. Its underlying hard problem (called the "discrete logarithm problem," or its elliptic-curve equivalent) is a different mathematical problem from RSA's, but is similarly efficiently solvable by the same category of quantum algorithm - so it is equally, completely quantum-broken, not merely weakened.
- **DSA and ECDSA (and Ed25519, a modern, widely-used variant)** - digital-signature algorithms: a way to produce a signature over a message that anyone can verify came from the true owner of a specific private key, without that owner having to be present or trust the verifier. These rely on the same family of underlying hard mathematical problems as Diffie-Hellman above, and are equally completely broken by the same quantum algorithm.

Symmetric cryptography, as a concept

Symmetric algorithms solve a different, simpler-sounding problem: if two parties *already* share one secret key (however they obtained it - often via one of the public-key methods above), how do they use that one shared key to scramble and unscramble a large amount of data quickly? Unlike public-key cryptography, there is no separate "public" and "private" half here - the same single key both locks and unlocks the data, which is why both parties must already share it in advance.

- **AES** - the current, modern, industry-standard symmetric algorithm, and the one this product recommends as the safe target for anything being migrated away from an older cipher. It comes in a few different key-size variants; a larger key size provides a larger safety margin. **Why it's only weakened, not broken:** a quantum computer can run a different, more limited quantum algorithm against symmetric ciphers, which effectively halves how strong a given key

size feels in practice - meaning what used to require an astronomically large amount of guessing now requires a much smaller (but still very large) amount of guessing. The practical fix is simply to use a larger key size to begin with, not to abandon the algorithm - this is the essential distinction described fully in Section 8.

- **DES and its successor Triple-DES** - older symmetric algorithms that are already considered broken today by entirely ordinary, non-quantum means - their key sizes are now small enough that a determined attacker with modern conventional hardware can already succeed, with no quantum computer required at all.
- **Blowfish** - another older symmetric algorithm with a specific structural weakness: the small size of the individual chunks of data it processes at once makes it vulnerable to a particular known attack once a large enough amount of data has been encrypted under the same single key.
- **RC4** - an older, different style of symmetric algorithm (it processes data one small piece at a time in a continuous stream, rather than in fixed-size chunks) with well-documented statistical weaknesses that make it unsafe for real security use today, again independent of any quantum computer.

Hash functions, as a concept

A hash function takes any input - a password, a file, a message of any size - and produces a short, fixed-size "fingerprint" of it. This fingerprint should be effectively impossible to reverse back into the original input, and even a tiny change to the input should produce a completely different-looking fingerprint. This makes hash functions useful for things like verifying a file hasn't been tampered with, or storing a representation of a password without storing the password itself.

- **MD5 and MD2** - older hash functions that are now considered completely broken, because it has become practical to deliberately construct two different inputs that produce the exact same fingerprint (called a "collision") - which defeats the entire purpose of a fingerprint being unique.
- **SHA-1** - a later hash function that was, for a long time, considered a safe replacement for MD5, but has since also had practical collision attacks demonstrated against it, and is now considered broken for security purposes.
- **SHA-256 and its larger variants** - the current, modern, industry-standard hash functions, still considered safe and collision-resistant today, including against the more limited quantum-weakening effect described above (Grover's algorithm), which reduces their effective strength somewhat but not to a genuinely dangerous degree at their current sizes.

The recommended replacements (post-quantum cryptography)

Post-quantum cryptography doesn't mean "cryptography that runs on a quantum computer" - it means the opposite: cryptography specifically designed to remain secure *against* an attacker who has a quantum computer, while still running on completely ordinary, everyday computers exactly like today's algorithms do. These replacement algorithms are built around entirely different underlying mathematical problems - ones that, as far as anyone currently knows, are not efficiently solvable by any quantum algorithm, unlike the factoring and discrete-logarithm problems that RSA, Diffie-Hellman, and ECDSA all depend on.

- **The recommended key-exchange replacement** is built around a mathematical problem involving highly structured multi-dimensional grids of numbers (broadly referred to as "lattice-based" cryptography) - a completely different mathematical foundation from anything a quantum computer's known algorithms are effective against.
- **The recommended digital-signature replacement** is built on that same general mathematical family, adapted specifically for producing and verifying signatures rather than establishing a shared secret.
- **A second, more conservative signature option** exists as a deliberate fallback, built on an entirely different mathematical foundation again (based purely on the properties of hash functions, described above) - offered specifically for organizations who want extra assurance by not depending on the newer lattice-based mathematics at all.
- **"Hybrid" recommendations** - in several cases, the product recommends running an old, familiar algorithm *and* a new, quantum-safe one together, side by side, during a transition period. The practical benefit is that the connection remains exactly as secure as before if the new algorithm somehow turns out to have an unexpected weakness, while still gaining real, immediate protection against the quantum-computing threat today - a commonly recommended, safety-first way to migrate.

29. CORE TERMINOLOGY YOU SHOULD KNOW

A working glossary, grouped by topic, for the cryptography, security, and product-specific vocabulary that comes up constantly when discussing this product.

Cryptography fundamentals

- **Encryption / Decryption** - scrambling readable information into an unreadable form (encryption), and reversing that process back into readable form using the correct key (decryption).
- **Key** - a piece of secret (or, for public-key cryptography, partly public) information that controls exactly how a specific piece of data gets scrambled or unscrambled. Without the correct key, reversing the scrambling should be practically impossible.
- **Key size / key length** - how large that key is, usually measured in bits. Generally, a larger key size means dramatically more possible keys an attacker would have to try, making an ordinary brute-force guessing attack proportionally harder.
- **Symmetric cryptography** - encryption where the exact same single key is used both to lock and to unlock the data; both parties must already share that one key.
- **Asymmetric (public-key) cryptography** - encryption where two mathematically related but different keys are used - one is shared publicly, the other kept strictly private - solving the problem of secure communication between parties who've never met.
- **Digital signature** - a way of using a private key to produce a proof, attached to a message, that only the true owner of that private key could have produced - and that anyone holding the matching public key can independently verify, without needing to trust anyone else.
- **Hash function** - a function that turns any input into a short, fixed-size fingerprint, designed so that reversing it or finding two different inputs with the same fingerprint should both be practically impossible.
- **Key exchange** - a method that lets two parties agree on a shared secret key over a channel that might be watched by an eavesdropper, without ever directly sending that secret itself across the channel.
- **Cipher** - a general term for an algorithm used to perform encryption and decryption.
- **Block cipher vs. stream cipher** - a block cipher processes data in fixed-size chunks at a time; a stream cipher processes data continuously, one small piece at a time. Each has different practical strengths and weaknesses depending on how it's used.
- **Salt** - random data mixed in before hashing something (very commonly a password), specifically so that the same input never produces the same fingerprint twice, which defeats a whole category of pre-computed lookup-table attacks.
- **Nonce / Initialization Vector (IV)** - a value used once, alongside a key, to make sure that encrypting the exact same data twice with the exact same key still produces different-looking scrambled output each time.
- **MAC (Message Authentication Code)** - a short value, computed using a shared secret key, that proves both that a message hasn't been tampered with *and* that it genuinely came from someone who knows that shared key - combining the integrity-checking idea of a hash function with the trust of a shared secret.
- **KDF (Key Derivation Function)** - a function specifically designed to turn something weaker (like a human-memorable password) into a strong, properly-sized cryptographic key, deliberately in a slow, resource-intensive way specifically to make large-scale guessing attacks impractical.

Public-key infrastructure and certificates

- **PKI (Public Key Infrastructure)** - the overall system of trust that lets public keys be reliably tied to a real, verified identity, so a stranger's public key can actually be trusted to belong to who it claims to.
- **Certificate (X.509)** - a digitally signed document that binds a specific public key to a specific identity (like a website's domain name), vouched for by a trusted third party.
- **Certificate Authority (CA)** - the trusted third party that actually issues and signs certificates, effectively vouching "we've verified this public key really does belong to this identity."
- **TLS / SSL** - the security protocol that protects most everyday web traffic, built on top of exactly this certificate and public-key infrastructure.

Quantum computing and post-quantum cryptography

- **Qubit** - the basic unit of information in a quantum computer, capable of representing information in fundamentally different ways than an ordinary computer's basic unit (a bit), which is what gives certain quantum algorithms their extraordinary speed advantage for specific kinds of mathematical problems.
- **Shor's algorithm** - a specific quantum algorithm that can efficiently solve the exact mathematical problems that RSA, Diffie-Hellman, and elliptic-curve cryptography all depend on - which is precisely why those algorithms are considered completely broken by a sufficiently capable quantum computer, not merely weakened.
- **Grover's algorithm** - a different quantum algorithm that provides a smaller, more general speed advantage for brute-force searching problems - relevant to symmetric ciphers and hash functions, where its effect is to roughly halve their effective strength rather than break them outright.
- **CRQC (Cryptographically Relevant Quantum Computer)** - the industry's shorthand for a quantum computer that is actually powerful and reliable enough to run these attacks against real-world cryptography - which does not exist yet, and whose arrival timeline is genuinely uncertain, not a settled fact.
- **Harvest-Now-Decrypt-Later** - the specific, present-day risk that an adversary is already capturing and storing today's encrypted data, intending to decrypt it retroactively once a capable quantum computer eventually exists - meaning some data is at risk today, even though the technology to actually break it doesn't exist yet.
- **Post-quantum cryptography (PQC)** - a newer generation of cryptographic algorithms, built on entirely different mathematical foundations, specifically designed to remain secure even against an attacker equipped with a quantum computer, while still running on ordinary computers today.
- **Crypto-agility** - an organization's general ability to actually swap out a cryptographic algorithm for a different one across its systems relatively quickly and painlessly, without a slow, high-risk, years-long rebuild - a real organizational capability that migration planning tools like this product are meant to support and measure.
- **Hybrid cryptography** - deliberately using an old, trusted algorithm and a new, quantum-safe algorithm together during a transition period, so security never regresses even if something unexpected is later found wrong with the new algorithm.

General security and threat terminology

- **Vulnerability** - a genuine weakness in a system that could realistically be exploited to cause harm.
- **Threat model** - a structured way of thinking through who might realistically want to attack a system, how they might try, and what the real consequences would be - used throughout this document's own security analysis.
- **Attack surface** - the total set of all the different ways a system could realistically be reached or attacked; a smaller, simpler attack surface is generally easier to defend well.
- **Least privilege** - the security principle that anything (a person, a program, a piece of code) should only ever be given the absolute minimum level of access it genuinely needs to do its job, and nothing more.
- **IDOR (Insecure Direct Object Reference)** - a specific, common category of flaw where a system trusts an identifier a user directly supplies (like an item's ID number in a web address) without properly re-checking that the user is actually allowed to access that specific item - this product was specifically checked for, and found not to have, this exact flaw (Section 13-14).
- **Sandboxing / isolation** - deliberately running an untrusted piece of work inside a strictly limited, contained environment, specifically so that even if something inside it turns out to be hostile, it cannot reach or affect anything outside that container.
- **Rate limiting** - deliberately capping how many times something can be requested within a given period of time, specifically to prevent a system from being accidentally or deliberately overwhelmed.
- **Defense in depth** - the practice of layering multiple, independent security protections on top of each other, so that a single failure or gap in any one layer doesn't, by itself, lead to a full breach.

Product-specific terminology

- **CBOM (Cryptographic Bill of Materials)** - a standardized, exportable inventory listing every cryptographic algorithm found in a system, similar in spirit to a nutrition label, but for cryptography rather than food ingredients.

- **SBOM (Software Bill of Materials)** - the broader, more general concept a CBOM is closely related to: a complete inventory of all the software components and dependencies that make up a system. A CBOM is a specialized version of this same idea, focused specifically on cryptography rather than software components generally.
- **Evidence** - this product's term for one single, specific real finding - a particular file, a particular line, and the specific piece of cryptography that was actually found there, kept as a permanent record and never fabricated or guessed.
- **Workspace** - the fully isolated space that holds one organization's entire set of connected projects, scan results, and history - never visible to, or reachable by, any other organization.
- **Discovery** - this product's term for the overall process of scanning a project and finding real cryptography inside it.
- **Normalization** - the step where raw, differently-worded findings (which might describe the same underlying algorithm in several different ways depending on how it was written in the original code) get matched to one single, consistent, canonical name.
- **Canonical asset** - the single, de-duplicated, authoritative entry representing one specific algorithm in the inventory, even if real evidence of it was found in many different places.
- **Blast radius** - how far the impact of a single specific finding actually reaches - which projects, which files, and what else lives alongside it - so a team can understand the true scope of fixing it before they start.
- **Migration status** - where a specific finding currently sits in the five-stage journey from first being found to being genuinely, provably fixed (Assessed, Planned, In Development, Testing, Fully Migrated), described fully in Section 2.
- **Verification** - the step that turns "we believe this is fixed" into "we have fresh, real evidence this is actually fixed," by genuinely re-scanning the real project and confirming the old algorithm is truly gone before marking anything as resolved.
- **Quantum Readiness Score** - the single overall number summarizing an organization's whole quantum-preparedness posture, always shown broken down into its real underlying components rather than as an unexplained bare number.
- **Composite risk** - the single, final risk level assigned to one specific finding, after combining both its classical (today's-computer) risk and its quantum risk together using the priority rules described in Section 7.

TOP 10 THINGS THIS REVIEW COULD NOT FULLY RESOLVE

1. The exact final network security setup once the product is actually deployed live is a hosting decision, not something a review of the product itself can fully confirm on its own.
2. Exactly what each outside AI provider does with the (non-code) information sent to them, once it reaches their own systems, is governed by their own policies, not something this product can fully verify from the inside.
3. No single, formal, published accuracy number (a measured "this correctly finds X% of real cryptography") currently exists to point to, even though real testing clearly happens.
4. There is a piece of code, mentioned nowhere in the visible product, that appears built for reading a live website's security certificate directly - it's unclear whether this was an intentionally paused feature or something left over from earlier exploration.
5. The exact real-world production setup (how the interface and backend are actually run together in a live environment) wasn't something this review could fully confirm from the product's own code alone.
6. Whether there has ever been an accidental slip in the project's history where a secret key was briefly exposed before being caught was not something a full historical review covered - only the current, present-day state was confirmed to be clean.
7. Some parts of the interface use the word "organization" in a way that seems to suggest more than the current strict one-person-per-workspace model actually supports - worth the team clarifying their own language before presenting, so nobody is caught off guard by the mismatch.
8. Whether the current set of specific security pattern-matching rules is considered "enough for now" or actively expected to keep growing wasn't clearly stated anywhere.
9. How the 3D relationship visualization would actually behave with a very large number of items on screen at once wasn't something this review could observe directly, only reason about.
10. Whether the non-functional example settings shown in the workspace settings screen are meant to preview genuinely planned near-term features, or are simply illustrative placeholders with no specific timeline - worth the team

clarifying internally, so nobody is caught off guard by a question about them during a live demonstration.

This document explains ECDAT in plain, theoretical language, grounded entirely in what the real, working product actually does. Nowhere does it overstate a capability that doesn't exist, and nowhere does it hide one that does.